

Lab 04 · El archivador

Curso de Spring Boot · Servicio de Impuestos Internos · 2026

90 minutos · Spring Boot 4.1.0 · Java 25 (Temurin) · PostgreSQL 16 embebido

Resumen

Que una clase Java y una tabla de la base de datos son la misma cosa, y que el SQL lo escribe otro. Vas a ver en la consola cada `INSERT`, cada `SELECT` y cada `UPDATE` que tú no escribiste.

Índice

1. Antes de empezar	2
1.1. Qué vas a lograr	2
1.2. Qué necesitas tener listo	2
1.3. Cómo copiar el código de esta guía	3
1.4. La puesta a punto	3
2. El caso	3
2.1. El archivador, que es la metáfora de este laboratorio	3
3. Los pasos	3
3.1. Paso 1 · Arrancar, y ver la base viva	3
3.2. Paso 2 · La ficha	5
3.3. Paso 3 · El archivista	8
3.4. Paso 4 · Guardar, y ver aparecer el número	9
3.5. Paso 5 · Buscar, con una condición y con dos	10
3.6. Paso 6 · La sorpresa: actualizar sin llamar a guardar	11
3.7. Paso 7 · Lo mismo, por HTTP	12
4. Lo que aprendiste	14
5. Para profundizar	15
6. Antes de cerrar	15

1 Antes de empezar

1.1 Qué vas a lograr

Hasta el Lab 03 todo lo que guardabas vivía en una lista dentro del programa y **desaparecía al apagarlo**. Hoy deja de desaparecer.

Vas a escribir una clase con cinco anotaciones y, sin escribir una línea de SQL, vas a poder guardar, buscar, filtrar por dos condiciones, actualizar y borrar. Y sobre todo: **vas a ver el SQL que se genera**, línea por línea, en tu consola. La sorpresa del laboratorio es el paso 7, donde algo se guarda **sin que llames a guardar**.

1.2 Qué necesitas tener listo

Requisito	Cómo lo compruebas	Qué tiene que salir
Los labs 00 a 03 hechos	Sabes arrancar, pedir dependencias y devolver errores	—
Estar en la carpeta del lab Nada más	cd labs/lab-04-jpa/practica No hay que instalar PostgreSQL	El cd no da error La base viaja dentro del repositorio

La base de datos es PostgreSQL de verdad, no una imitación en memoria. Se descomprime sola en una carpeta `.datos-pg/` la primera vez y arranca como un proceso hijo de la

aplicación. No hay que instalar nada, ni tener Docker.

Por eso **el primer arranque tarda más**: está creando la base. Los siguientes son rápidos, y los datos siguen ahí.

1.3 Cómo copiar el código de esta guía

Al copiar de un PDF se pierden los espacios del principio de línea, y a veces una línea larga se parte en dos. Con Java no importa —el compilador ignora la sangría—; si una línea se parte, el editor te la marca y basta con unirla. El código completo está en `labs/lab-04-jpa/solucion/`.

1.4 La puesta a punto

```
cd labs/lab-04-jpa/practica
./mvnw spring-boot:run
```

La aplicación **arranca la base, corre las demos, imprime, y se queda escuchando** en el 8099. Se para con `Ctrl+C`.

Párala siempre con Ctrl+C. Si cierras la terminal de golpe, el PostgreSQL puede sobrevivir a la aplicación y el siguiente arranque no encontrará su puerto libre. Los dos casos están en el «Si te atascas» del paso 1, con el mensaje exacto y el comando para arreglarlo.

2 El caso

La oficina de la DGT lleva sus observaciones **en un cuaderno**. Al cerrar por la tarde, el cuaderno se tira. Mañana se empieza otra vez en blanco.

2.1 El archivador, que es la metáfora de este laboratorio

La oficina compra un archivador.

Un archivador tiene **cajones**, y cada cajón guarda **fichas** de un mismo tipo: el cajón de las observaciones, el de los contribuyentes. Un cajón es **una tabla**; una ficha es **una fila**.

Y aquí está lo que hay que entender hoy: **tu clase Java ES la ficha**. No es una copia de la ficha, ni algo que se parece a la ficha. Es la misma cosa vista desde Java. Las anotaciones son sólo **las etiquetas que dicen a qué cajón va y qué va en cada casilla**.

Cada ficha lleva **un número correlativo** que no pone nadie a mano: lo pone el propio archivador al meterla. Eso es `@GeneratedValue`, y en el paso 3 lo vas a ver ocurrir.

Y hay **un archivista** que sabe usar el archivador: tú le dices «*tráeme las de Carolina*» y él sabe a qué cajón ir y cómo buscar. Tú no abres cajones. Ese archivista es **el repositorio**, y lo notable es que **no vas a escribirlo**: basta con decirle qué le vas a pedir.

3 Los pasos

3.1 Paso 1 · Arrancar, y ver la base viva

Qué vamos a hacer

Arrancar sin escribir nada, para ver que la base se levanta sola y que la tabla ya está creada.

Para entenderlo mejor

Comprobar que el archivador llegó, que tiene su cajón, y que está vacío esperando fichas.

El problema

Trabajar contra una base de datos suele exigir instalarla, configurarla, crear un usuario y una contraseña. Eso es media clase perdida antes de aprender nada, y en las máquinas del SII ni siquiera es posible: no hay permisos de administrador.

La alternativa, y por qué no

- **Una base en memoria** tipo H2: arranca al instante y no necesita nada. Se descarta porque su SQL **no es el de PostgreSQL**: aprenderías un dialecto que no vas a usar, y las diferencias aparecen justo en lo que importa.
- **PostgreSQL instalado en la máquina**: es lo real, y aquí es imposible.
- **PostgreSQL embebido**, que es lo de aquí: PostgreSQL **de verdad**, versión 16, que viaja como una dependencia más y arranca como proceso hijo. El SQL que veas es el que verías en producción.

Se corre

```
./mvnw spring-boot:run
```

Lo que vas a ver

Entre el ruido del arranque, la línea de Flyway creando la tabla, y después las demos. La estructura de la tabla la crea esto, que ya viene escrito:

```
CREATE TABLE observacion (  
  id      BIGSERIAL      PRIMARY KEY,  
  
  texto   VARCHAR(500) NOT NULL,  
  autor   VARCHAR(100) NOT NULL,  
  
  fecha   DATE           NOT NULL  
);
```

Vas bien si...

La aplicación arranca sin errores y la consola muestra las demos numeradas. La primera vez tarda bastante más; es normal.

Si te atascas

1 · EL PUERTO 55433 YA ESTA OCUPADO

```
=====
EL PUERTO 55433 YA ESTA OCUPADO
=====
```

Ahi es donde este proyecto levanta su PostgreSQL, asi que no puede arrancar.

Lo mas probable: quedo un PostgreSQL vivo de una corrida anterior de ESTE mismo proyecto. Al cerrar con Ctrl+C, o al cerrar la terminal de golpe, el motor a veces sobrevive al programa que lo levanto.

N0 es un error de tu codigo.

```
=====
```

Es el error más común de todos los labs con base de datos, y no es culpa tuya. El propio mensaje trae el comando. En macOS o Linux:

```
lsof -ti:55433 | xargs kill -9
```

En Windows: netstat -ano | findstr :55433 y después taskkill /F /PID <el PID>.

2 · ESTE MISMO PROYECTO YA ESTA CORRIENDO

```
ESTE MISMO PROYECTO YA ESTA CORRIENDO
=====
```

El archivo .datos-pg/epg-lock esta tomado por otra aplicacion viva, asi que este arranque no puede usar la base.

Ese candado lo retiene el PROGRAMA, no PostgreSQL: sigue puesto aunque su motor ya no este.

Son dos candados distintos y conviene no confundirlos. El primero lo tiene PostgreSQL y se suelta al matarlo; **éste lo tiene la aplicación Java**, y sigue puesto aunque hayas matado el PostgreSQL a mano. Lo tienes arrancado en otra terminal: ve a ella y Ctrl+C. Si no la encuentras:

```
lsof -t .datos-pg/epg-lock | xargs kill -9
```

3 · El arranque se queda parado mucho rato la primera vez.

Está descomprimiendo PostgreSQL y creando la base. Solo pasa una vez. Déjalo.

3.2 Paso 2 · La ficha

Qué vamos a hacer

Escribir la clase que se corresponde con la tabla.

Para entenderlo mejor

Rotular la ficha: **a qué cajón pertenece**, cuál es **su número**, y qué va en **cada casilla**. Nada más. La ficha no sabe buscar ni guardarse: sólo dice qué es.

El problema

Entre una fila de la base y un objeto de Java hay que traducir en las dos direcciones, y esa traducción escrita a mano —leer columna por columna, construir el objeto, y al revés para guardar— son decenas de líneas por tabla que no enseñan nada y que se rompen en cuanto alguien añade una columna.

La alternativa, y por qué no

- **JDBC a mano**: abres conexión, escribes el SQL, recorres el `ResultSet` y construyes el objeto. Es lo que hay por debajo de todo, y da control total. Cuesta unas cincuenta líneas por tabla y cada una es una oportunidad de equivocarse.
- **Un `JdbcTemplate` o similar**: quita la fontanería y deja el SQL en tus manos. Sigue siendo buena opción cuando las consultas son el corazón del sistema.
- **JPA con anotaciones**, que es lo de aquí: describes **la correspondencia una vez** y el SQL de las operaciones corrientes lo escribe Hibernate. Se paga con tener que entender qué genera — y por eso este lab enciende `show-sql` desde el primer minuto: **aquí no hay magia, hay SQL que vas a leer**.

Se pega

Archivo `practica/src/main/java/cl/dgt/jpa/entities/Observacion.java` — el archivo entero:

```
package cl.dgt.jpa.entities;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

import java.time.LocalDate;

@Entity
@Table(name = "observacion")
public class Observacion {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 500)
    private String texto;

    @Column(nullable = false, length = 100)
    private String autor;

    @Column(nullable = false)
    private LocalDate fecha;

    protected Observacion() {
    }
}
```

```

public Observacion(String texto, String autor, LocalDate fecha) {
    this.texto = texto;
    this.autor = autor;
    this.fecha = fecha;
}

public Long getId() { return id; }
public String getTexto() { return texto; }
public String getAutor() { return autor; }
public LocalDate getFecha() { return fecha; }

public void setTexto(String texto) { this.texto = texto; }

@Override
public String toString() {
    return "Observacion{id=%d, texto='%s', autor='%s', fecha=%s}"
        .formatted(id, texto, autor, fecha);
}
}

```

Cinco cosas, y cada una tiene su casilla en la ficha:

- `@Entity` — esto es una ficha. Sin ella, las demás anotaciones no significan nada.
- `@Table(name = "observacion")` — a qué cajón va.
- `@Id` — cuál es el número de la ficha.
- `@GeneratedValue(IDENTITY)` — el número lo pone el archivador, no tú.
- `@Column(nullable = false, length = 500)` — qué admite cada casilla.

Fíjate en que NO es un record, a diferencia de los DTO de los labs anteriores. JPA necesita poder construir el objeto vacío y rellenarlo casilla a casilla, y un record es inmutable: no se deja. Ésa es la razón del constructor `protected` sin argumentos.

Vas bien si...

La aplicación arranca. Si la tabla y la clase no cuadraran, `ddl-auto: validate` lo diría al arrancar en vez de fallar más tarde.

Si te atascas

1 · Schema-validation: missing column [...]

La clase y la tabla no coinciden: escribiste mal un nombre, o le pusiste una casilla que el cajón no tiene. El mensaje dice cuál. **Este error es un regalo**: lo da al arrancar, no en producción.

2 · No identifier specified for entity

Falta el `@Id`. Toda ficha necesita su número.

3 · cannot find symbol: class Entity

Faltan los `import` de `jakarta.persistence`. Ojo: son `jakarta`, no `javax` — el nombre cambió, y casi todo lo que hay en internet todavía dice `javax`.

3.3 Paso 3 · El archivista

Qué vamos a hacer

Declarar una interfaz vacía y recibir gratis quince métodos.

Para entenderlo mejor

Contratar al archivista. No hay que enseñarle a abrir cajones: **basta con decirle qué le vas a pedir**, y él sabe hacerlo.

El problema

Guardar, buscar por id, listar, borrar y contar son las mismas cinco operaciones en todas las tablas de todos los proyectos del mundo. Escribirlas una vez por tabla es trabajo repetido que no distingue a nadie.

La alternativa, y por qué no

- **Escribir la clase de acceso a datos a mano**, con su SQL. Necesario cuando la consulta es rara; absurdo para «tráeme el de este id».
- **CrudRepository**: da lo básico. **PagingAndSortingRepository**: añade orden y páginas.
- **JpaRepository**, que es lo de aquí y hereda de los dos: añade además cosas propias de JPA como `flush()` y `saveAll()`. Se elige porque es lo que se encuentra en cualquier proyecto real, y porque tener de más no cuesta nada.

Se pega

Archivo `practica/src/main/java/cl/dgt/jpa/repositories/ObservacionRepository.java` — entero:

```
package cl.dgt.jpa.repositories;

import cl.dgt.jpa.entities.Observacion;
import org.springframework.data.jpa.repository.JpaRepository;

import java.time.LocalDate;
import java.util.List;

public interface ObservacionRepository extends JpaRepository<Observacion, Long> {

    List<Observacion> findByAutor(String autor);

    List<Observacion> findByAutorAndFechaAfter(String autor, LocalDate fecha);

    long countByAutor(String autor);
}
```

Es una interfaz y no tiene implementación. Nadie escribe la clase que la cumple: Spring Data la fabrica al arrancar **leyendo el nombre de cada método**. `findByAutor` se convierte en ... `where autor = ?`. `findByAutorAndFechaAfter`, en ... `where autor = ? and fecha > ?`.

Vas bien si...

Arranca sin errores. Si te hubieras equivocado en el nombre de un método —`findByAutorr`—, la aplicación **no arrancaría**: Spring Data valida los nombres al construir el repositorio.

Si te atascas

1 · No property 'autorr' found for type 'Observacion'

Un nombre de método no cuadra con ninguna casilla de la ficha. El mensaje dice cuál cree que buscabas. Es un error de arranque, no de ejecución: se descubre al segundo, no en producción.

3.4 Paso 4 · Guardar, y ver aparecer el número

Qué vamos a hacer

Guardar la primera ficha y mirar el INSERT que tú no escribiste.

Para entenderlo mejor

Meter la ficha en el cajón. **Antes de meterla no tiene número; al meterla, el archivador se lo pone.**

Se pega

En `practica/src/main/java/cl/dgt/jpa/demos/DemosJpa.java`:

```
public void guardar() {
    seccion(1, "GUARDAR · save()");

    repositorio.deleteAll();

    Observacion nueva = new Observacion(
        "Revisión anual sin hallazgos.", "Carolina", LocalDate.of(2026, 3, 10));
    System.out.println(" antes de guardar -> id = " + nueva.getId());

    Observacion guardada = repositorio.save(nueva);
    System.out.println(" después de guardar -> id = " + guardada.getId());
    this.primerId = guardada.getId();

    repositorio.save(new Observacion(
        "Solicita certificado de situación.", "Carolina", LocalDate.of(2026, 8,
        ↵ 1)));
    repositorio.save(new Observacion(
        "Diferencias en el F29 de julio.", "Ignacio", LocalDate.of(2026, 7, 15)));
    System.out.println(" (guardadas 2 más, para las demos siguientes)");
}
```

Lo que vas a ver

```
=== 1 · GUARDAR · save() ===
antes de guardar -> id = null
```

```
Hibernate:
    insert
    into
        observacion
        (autor, fecha, texto)
    values
        (?, ?, ?)
después de guardar -> id = 1
```

Míralo despacio, porque es el paso entero:

- **antes de guardar** -> `id = null`. El objeto existe en Java y todavía no es una fila.
- **El insert no menciona el id**. No hace falta: lo pone la base.
- **después de guardar** -> `id = 1`. El objeto que tienes en la mano **ya sabe su número**, y nadie se lo asignó desde Java.

Los ids pueden no empezar en 1 en tu máquina. Si ya habías arrancado antes, el contador de la base siguió avanzando. Lo que importa es que pase de `null` a un número.

Vas bien si...

Ves el insert en la consola, y el id pasa de `null` a un número.

Si te atascas

1 · No ves ningún SQL en la consola.

Falta `spring.jpa.show-sql: true` en `application.yml`, o lo cambiaste sin querer. En este lab el SQL es el contenido: sin verlo, no se está viendo nada.

2 · ids duplicated o el insert menciona el id.

Le pusiste `@GeneratedValue` mal, o se lo quitaste. Con `IDENTITY`, el id lo pone la columna `BIGSERIAL` de la tabla.

3.5 Paso 5 · Buscar, con una condición y con dos

Qué vamos a hacer

Pedirle al archivista las fichas de un autor, y después las de un autor a partir de una fecha.

Para entenderlo mejor

«*Tráeme las de Carolina*» y «*tráeme las de Carolina posteriores a junio*». **Tú no dices cómo buscarlas.** Dices qué quieres, y el nombre de lo que pides ya lo describe.

Se pega

```
public void buscarConDosCondiciones() {
    seccion(5, "DOS CONDICIONES · findByAutorAndFechaAfter()");

    LocalDate corte = LocalDate.of(2026, 6, 1);
    List<Observacion> recientes = repositorio.findByAutorAndFechaAfter("Carolina",
        ↪ corte);
}
```

```

System.out.println(" autor = Carolina y fecha > " + corte + " -> " +
    ↪ recientes.size());
recientes.forEach(o -> System.out.println("    " + o));
}

```

Lo que vas a ver

```

=== 5 · DOS CONDICIONES · findByAutorAndFechaAfter() ===
Hibernate:
    select
        o1_0.id, o1_0.autor, o1_0.fecha, o1_0.texto
    from
        observacion o1_0
    where
        o1_0.autor=?
        and o1_0.fecha>?
autor = Carolina y fecha > 2026-06-01 -> 1
Observacion{id=2, texto='Solicita certificado de situación.', autor='Carolina',
    ↪ fecha=2026-08-01}

```

El **and** de la consulta salió del **And** del nombre del método. Nadie escribió ese SQL.

Vas bien si...

La consulta que sale en consola tiene un **where** con dos condiciones, y el resultado es una sola observación.

Si te atascas

1 · No property found for type al arrancar.

El nombre del método menciona una casilla que la ficha no tiene, o está mal escrita. Recuerda que las casillas se llaman como los campos de la clase, no como las columnas de la tabla.

2 · Devuelve más filas de las que esperas.

After es **estrictamente mayor**. Si quieres incluir la fecha, es `findByAutorAndFechaGreaterThanEqual`.

3.6 Paso 6 · La sorpresa: actualizar sin llamar a guardar

Qué vamos a hacer

Cambiar el texto de una observación **sin llamar a `save()`**, y ver que se guarda igual.

Para entenderlo mejor

Mientras la ficha está **sobre la mesa del archivista** —dentro de una transacción—, él la vigila. Si le cambias algo, al cerrar la carpeta **la copia al cajón sin que se lo pidas**. No es magia: es que comparó cómo estaba y cómo quedó.

El problema

Si cada cambio exigiera acordarse de llamar a `save()`, olvidarlo sería un fallo silencioso: el programa haría lo correcto en memoria y no lo guardaría.

La alternativa, y por qué no

Hacerlo explícito —modificar y llamar a `save()`— es más obvio de leer, y hay quien lo prefiere por eso. El precio de lo automático es que **hay que saber que existe**: modificar una entidad dentro de una transacción **la guarda**, quisieras o no. Es la causa de guardados accidentales cuando alguien toca un objeto «solo para leerlo».

Por eso este lab lo enseña ahora y no lo esconde.

Se pega

```
@Transactional
public void actualizar() {
    seccion(6, "ACTUALIZAR SIN save() · dirty checking");

    Observacion observacion = repositorio.findById(primerId).orElseThrow();
    System.out.println(" antes: " + observacion.getTexto());

    observacion.setTexto("Revisión anual: se detecta diferencia menor.");
    System.out.println(" después: " + observacion.getTexto());
    System.out.println(" NO llamamos a save(). El UPDATE aparece justo aquí abajo,");
    System.out.println(" cuando esta transacción se cierre:");
}
```

Busca el `save()` en ese método. No está.

Lo que vas a ver

Un update en la consola, disparado por el simple hecho de haber cambiado el objeto dentro de un método `@Transactional`.

Vas bien si...

Sale un update ... set texto=? en la consola y no hay ninguna llamada a `save()` en el código que lo provocó.

Si te atascas

1 · No sale ningún update.

Falta el `@Transactional` en el método. Sin transacción no hay nadie vigilando la ficha, y el cambio se queda solo en memoria.

3.7 Paso 7 · Lo mismo, por HTTP

Qué vamos a hacer

Poner una ventanilla delante del archivador: los mismos métodos, ahora desde fuera.

Para entenderlo mejor

El público no entra al archivo. Pide en **la ventanilla** del Lab 01, y el archivista trae.

Se pega

practica/src/main/java/cl/dgt/jpa/controllers/ObservacionController.java — el archivo entero, **borrando lo que había**:

```
package cl.dgt.jpa.controllers;

import cl.dgt.jpa.entities.Observacion;
import cl.dgt.jpa.repositories.ObservacionRepository;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/api/observaciones")
public class ObservacionController {

    private final ObservacionRepository repositorio;

    public ObservacionController(ObservacionRepository repositorio) {
        this.repositorio = repositorio;
    }

    @GetMapping
    public List<Observacion> listar(@RequestParam(required = false) String autor) {
        return (autor == null) ? repositorio.findAll() :
            ↪ repositorio.findByAutor(autor);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Observacion> porId(@PathVariable Long id) {
        return repositorio.findById(id)
            .map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<Observacion> crear(@RequestBody Observacion nueva) {
        return
            ↪ ResponseEntity.status(HttpStatus.CREATED).body(repositorio.save(nueva));
    }
}
```

```
}
```

Se corre

```
curl localhost:8099/api/observaciones
curl "localhost:8099/api/observaciones?autor=Carolina"
curl -i localhost:8099/api/observaciones/9999
curl -i -X POST localhost:8099/api/observaciones \
  -H 'Content-Type: application/json' \
  -d '{"texto":"Creada desde la guia.", "autor":"Rodrigo", "fecha":"2026-08-15"}'
```

Lo que vas a ver

```
[{"texto":"Solicita certificado de
↪ situación.", "autor":"Carolina", "fecha":"2026-08-01", "id":2},
{"texto":"Diferencias en el F29 de
↪ julio.", "autor":"Ignacio", "fecha":"2026-07-15", "id":3}]
```

HTTP/1.1 404 <- el id 9999 no existe

HTTP/1.1 201

```
{"texto":"Creada desde la guia.", "autor":"Rodrigo", "fecha":"2026-08-15", "id":4}
```

El 404 es para lo que servía el optional del paso 3: `findById` devuelve «puede que haya o puede que no», y el controlador convierte ese «no» en un 404.

Vas bien si...

Los cuatro `curl` responden: la lista, la lista filtrada, un 404 y un 201 con el id nuevo.

Si te atascas

1 · Todo da 404, incluso `/api/observaciones`.

La ruta lleva `/api` delante. Y comprueba el puerto: en practica/ es el **8099**.

2 · El POST da 400 o 500.

La fecha va en formato AAAA-MM-DD y entre comillas. Y no mandes id: lo pone la base.

4 Lo que aprendiste

1 · Una clase y una tabla son la misma cosa.

Cinco anotaciones describen la correspondencia, y con eso Hibernate escribe el SQL de todas las operaciones corrientes. No es magia: es SQL, y lo has estado leyendo en la consola todo el rato.

2 · El repositorio no se escribe: se declara.

Una interfaz que extiende `JpaRepository` llega con quince métodos hechos, y los que faltan se piden **escribiendo su nombre**. Si el nombre no cuadra con la ficha, la aplicación no arranca.

3 · Dentro de una transacción, modificar es guardar.

Cambiar un objeto gestionado lo guarda al terminar el método, sin `save()`. Es cómodo y es peligroso: hay que saberlo para no guardar sin querer.

4 · La base es de verdad, y por eso el SQL también.

PostgreSQL 16 corriendo como proceso hijo, sin instalar nada. Lo que ves en la consola es lo que verías en producción — que es justo lo que una base en memoria no te habría dado.

5 Para profundizar

- **Añade `findByTextoContaining(String trozo)`** al repositorio y llámalo. Mira el `like` que genera.
- **Quita el `@Transactional`** del método del paso 6 y vuelve a correr. ¿Sigue saliendo el `update`?
- **Cambia `length = 500` a `length = 50`** en el texto y arranca. Lee el error de validación: te está diciendo que la clase y la tabla dejaron de cuadrar.
- **Apaga la aplicación y vuelve a arrancarla** sin borrar `.datos-pg/`. Los datos siguen ahí. Ahora borra la carpeta y arranca: Flyway vuelve a crear la tabla desde cero.
- **Pon `spring.jpa.properties.hibernate.format_sql: false`** y mira la diferencia. Después déjalo como estaba.

6 Antes de cerrar

Párala con `Ctrl+C`. Es importante en este lab: si cierras la terminal de golpe, el PostgreSQL puede quedar vivo y el próximo arranque fallará con el mensaje del paso 1.

```
./mvnw clean
```

`clean` **no borra** `.datos-pg/`: tus datos siguen ahí para la próxima. Si quieres empezar de cero, borra esa carpeta a mano.

Lo que te llevas:

Una clase con `@Entity` es una fila de una tabla. Una interfaz que extiende `JpaRepository` es un archivista que ya sabe buscar. El SQL lo escribe Hibernate, y se puede leer en la consola.

Lo que queda pendiente, y abre el Lab 05: hoy la ficha estaba sola en su cajón. En la vida real una observación pertenece a un contribuyente, y un contribuyente tiene muchos trámites. **¿Cómo se apuntan dos fichas entre sí, y qué pasa cuando pides una y viene la otra detrás?**